Combinational Logic Design

Contemporary Digital Electronic Systems — Chapter 7

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Chapter
2. Design Procedure
3. Universal Gates
4. NAND-NAND Networks
5. Mixed Logic
6. NAND Gate ICs
7. Summary

Why This Chapter Matters

The earlier chapters each built one skill. This chapter combines them into a complete, repeatable design procedure: understand the problem, specify a truth table, minimise, draw the schematic, and verify. Every later design chapter follows this template.

After this chapter, you can:

1. Translate an English specification into a truth table.
2. Apply the combinational-design procedure.
3. Design a two-level NAND-only or NOR-only implementation.
4. Verify a circuit against its specification by simulation.

© Muním Zabidi

An Opening Question

If a combinational circuit's output depends only on its current inputs, what is it allowed to remember?

Answer: nothing. Change an input, wait t_{pd}, and the output follows. There is no past, no state and no clock.

Table of Contents

1 Why This Chapter

2 **Design Procedure**

3 Universal Gates

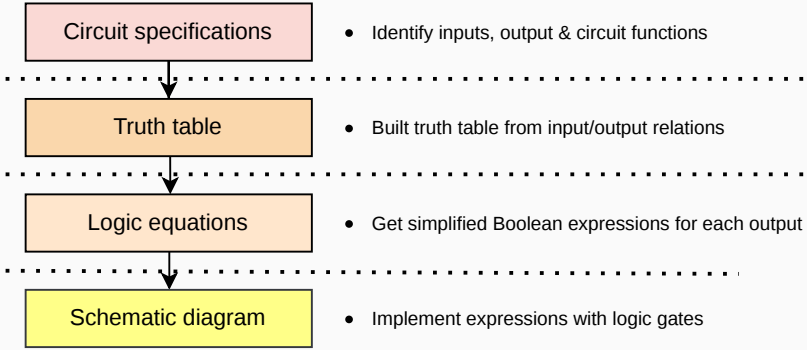
4 NAND-NAND Networks

5 Mixed Logic

6 NAND Gate ICs

7 Summary

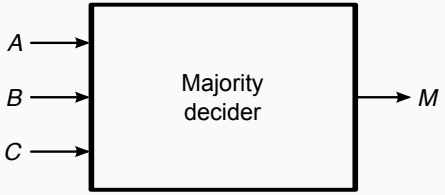
The Design Procedure



1. Specify: identify the inputs and outputs and the relationship between them.
2. Truth table: describe that relationship for every input combination.
3. Minimise: derive a simplified SOP for each output (K-map or Boolean algebra).
4. Implement with gates and verify in simulation.

© Munim Zabidi

Example 1: Majority Decider



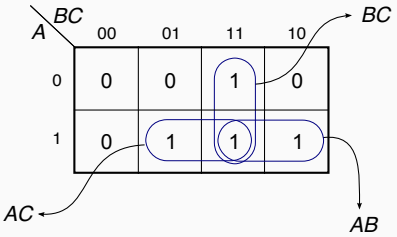
Step 1. Three inputs A, B, C; one output M that is 1 when two or three inputs are 1.

A	B	C	M
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Step 2. $M = 1$ for m_3, m_5, m_6, m_7.

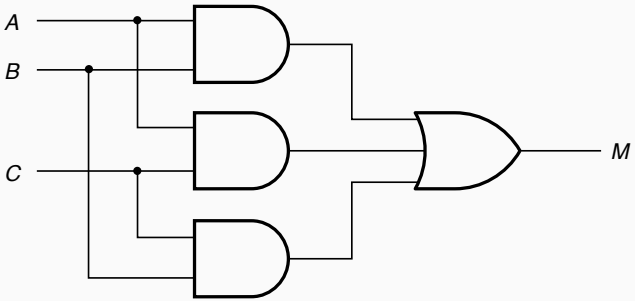
© Muním Zabidi

Majority Decider: Equation and Schematic



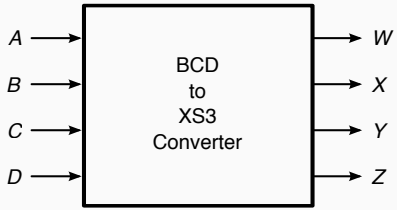
Step 3. Three overlapping pair loops; *ABC* is used in all three.

$$M = AB + AC + BC$$



Step 4. Each product term is a 2-input AND; a 3-input OR combines them.

Example 2: BCD to Excess-3 Converter

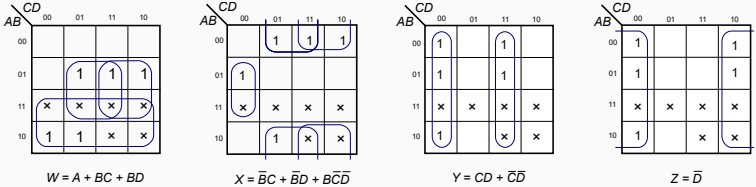


- 4 inputs *ABCD* (BCD), 4 outputs *WXYZ* (XS-3).
- Inputs 1010 to 1111 never occur: all outputs are don't cares.

A	*B*	*C*	*D*	*W*	*X*	*Y*	*Z*
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	×	×	×	×
1	0	1	1	×	×	×	×
1	1	0	0	×	×	×	×
1	1	0	1	×	×	×	×
1	1	1	0	×	×	×	×
1	1	1	1	×	×	×	×

© Munim Zabidi

BCD to XS-3: Four K-Maps



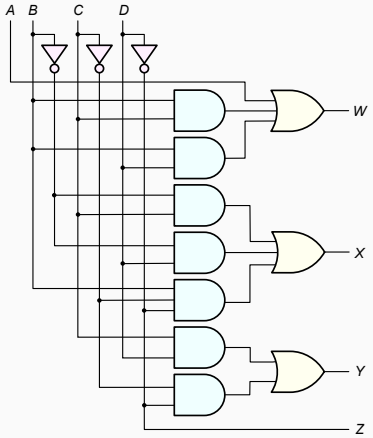
- One map per output bit. Include don't cares in a loop whenever it makes the loop larger.

Simplified equations

$W = A + B\overline{C} + BD$ $\quad X = \overline{B}C + \overline{B}D + B\overline{C}\,\overline{D}$

$Y = CD + \overline{C}\,\overline{D}$ $\quad Z = \overline{D}$

BCD to XS-3: Schematic



- The same AND–OR translation as the majority circuit: one gate per product term, one OR per output.
- Four outputs make the schematic larger, but no new technique is needed.

© Muním Zabidi

Table of Contents

1 Why This Chapter

2 Design Procedure

3 **Universal Gates**

4 NAND-NAND Networks

5 Mixed Logic

6 NAND Gate ICs

7 Summary

Why Universal Gates?

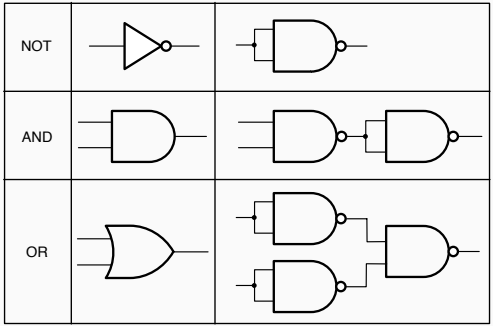
- Using as few gate types as possible reduces design complexity and cost, so ICs are built from NAND or NOR.
- De Morgan's theorems show why: $AB = \overline{\overline{A} + \overline{B}}$ and $A + B = \overline{\overline{A}\,\overline{B}}$.
- A NAND gate equals an OR gate with inverted inputs: $\overline{AB} = \overline{A} + \overline{B}$.



A	B	$\overline{A}$	$\overline{B}$	$\overline{AB}$	$\overline{A} + \overline{B}$
0	0	1	1	1	1
0	1	1	0	1	1
1	0	0	1	1	1
1	1	0	0	0	0

© Munim Zabidi

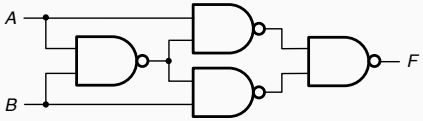
Building Gates from NAND



NOT from NAND

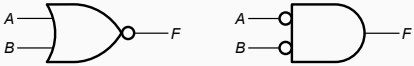
Tie the inputs together: $F = \overline{A \cdot A} = \overline{A}$ (idempotent law). Tying NOR inputs works the same way.

- With NOT available, NAND gates give AND and OR too.
- XOR can also be built from four NAND gates.



© Muním Zabidi

NOR Is Universal Too



A	B	$\overline{A}$	$\overline{B}$	$\overline{A+B}$	$\overline{A}\,\overline{B}$
0	0	1	1	1	1
0	1	1	0	0	0
1	0	0	1	0	0
1	1	0	0	0	0

NOR = AND with inverted inputs.

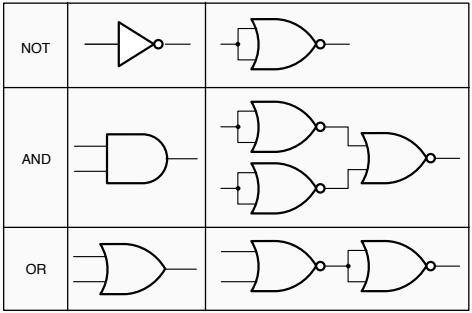


Table of Contents

1 Why This Chapter

2 Design Procedure

3 Universal Gates

4 **NAND-NAND Networks**

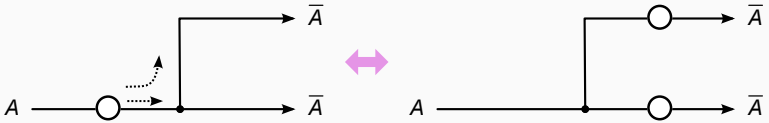
5 Mixed Logic

6 NAND Gate ICs

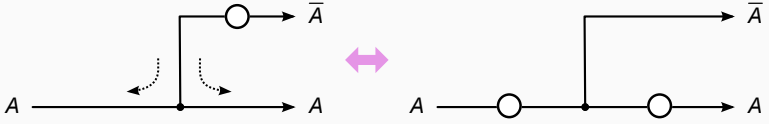
7 Summary

Bubble Pushing

- Start from the AND–OR–NOT schematic. Only AND and OR gates are converted; NOT gates stay.
- Add pairs of bubbles: two consecutive bubbles cancel.
- Slide bubbles along a wire until they meet a gate. Gates with bubbles become NAND (or NOR) gates.



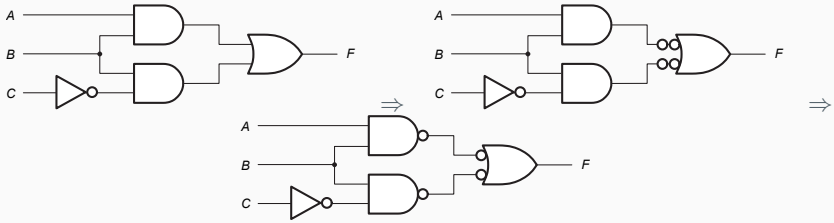
Pushing bubbles to the right



Pushing bubbles to the left

© Munim Zabidi

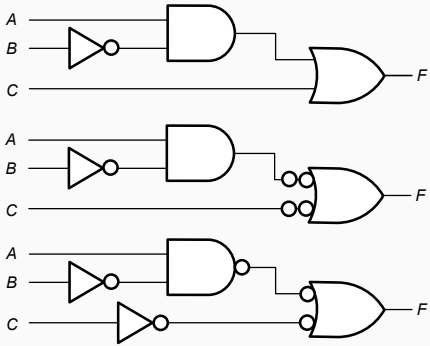
Example: $F = AB + B\overline{C}$



- Add double bubbles between the AND and OR gates.
- Slide the left bubble of each pair back to the AND output: the AND gates become NAND.
- The OR gate with inverted inputs is also a NAND, by De Morgan. The function is unchanged.

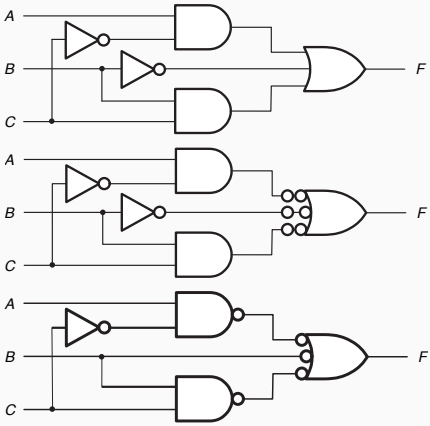
Examples: Extra Inverter, Cancelled Inverter

$F = A\overline{B} + C$



Two bubbles on the OR input need an added inverter.

$F = A\overline{C} + \overline{B} + BC$



Three bubbles on the middle input: two cancel and the inverter disappears.

Table of Contents

1 Why This Chapter

2 Design Procedure

3 Universal Gates

4 NAND-NAND Networks

5 **Mixed Logic**

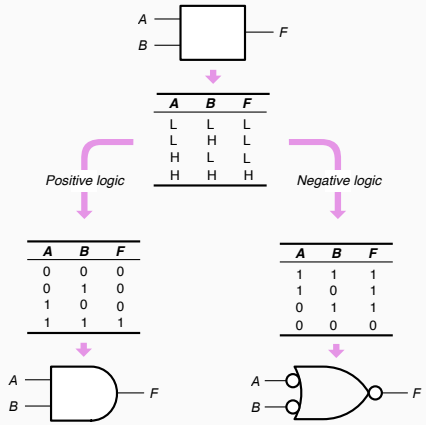
6 NAND Gate ICs

7 Summary

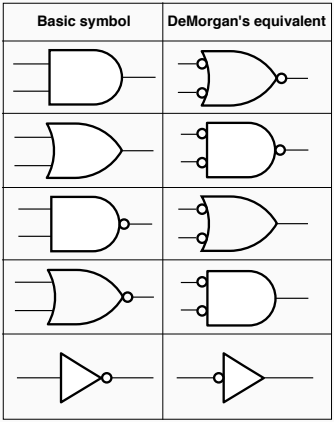
Positive and Negative Logic

- Positive logic: 1 is the higher voltage. Negative logic: 1 is the lower voltage.
- The same device is an AND in positive logic and an OR in negative logic.
- Mixed logic separates the *functional description* from the *physical implementation*.

AND (pos)	⇔	OR (neg)
OR (pos)	⇔	AND (neg)
NAND (pos)	⇔	NOR (neg)
NOR (pos)	⇔	NAND (neg)



De Morgan Equivalent Symbols



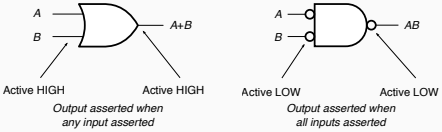
To get the alternative symbol

1. Invert all inputs and outputs: add a bubble where there is none, remove one where there is.
2. Switch the symbol: AND to OR, OR to AND. For NOT, move the bubble.

Example: NAND (AND with output bubble) becomes OR with bubbles on all inputs.

© Muním Zabidi

Choosing the Symbol; Active-Low Signals



Positive-logic OR Negative-logic AND

- Describe the function in words first ("output active when all inputs are active" is AND), then add bubbles for the signal polarity.

- Active-low signals reduce errors in noisy environments and are used for control signals between chips.
- Notation: $\overline{RESET}$, *RESETn*, *RESET**.
- Dual-meaning names: *READ*/$\overline{WRITE}$, *M*/$\overline{IO}$ (high / low meanings).

Table of Contents

1 Why This Chapter

2 Design Procedure

3 Universal Gates

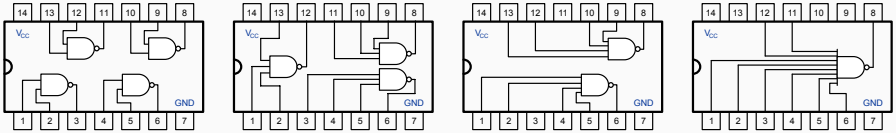
4 NAND-NAND Networks

5 Mixed Logic

6 **NAND Gate ICs**

7 Summary

NAND Gate ICs



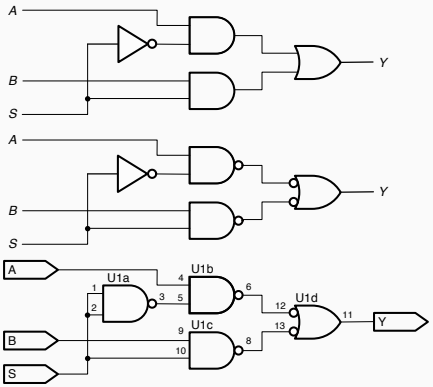
74x00 74x10 74x20 74x30

- NAND gates have long been sold in many IC types. These TTL chips are historical but still illustrate the principle.
- An equivalent function can be built with different gate mixes and different hardware.

© Munim Zabidi

Example: 2:1 Mux in One IC

$$Y = S'A + SB$$



Original (inverter, 2 AND, 1 OR = 3 ICs) → all NAND/NOT → one 74x00 with ports and pins numbered.

Is It Still Worth Doing?

- NAND-based design can cut ICs, wiring, board space and cost. It does not automatically cut power or delay.
- Modern systems use microcontrollers, CPLDs, FPGAs or highly integrated logic, not discrete TTL gates.
- NAND implementations remain a good teaching tool: they show universality.

Table of Contents

1 Why This Chapter

2 Design Procedure

3 Universal Gates

4 NAND-NAND Networks

5 Mixed Logic

6 NAND Gate ICs

7 **Summary**

Summary

- Design is: specify → truth table → minimise → draw → verify.
- A combinational circuit has no memory: its output is a pure function of its current inputs.
- An AND–OR–NOT schematic converts mechanically to NAND-only (or NOR-only) with De Morgan's laws and bubble pushing.
- Verify the final circuit against the specification by simulation.

Looking ahead

Chapter 8 introduces standard modules (decoders, encoders, multiplexers, demultiplexers) that you compose instead of designing from scratch.

© Munim Zabidi